Memory Forensics Writeup

Volatility3 — Banking Trojan, WannaCry Ransomware & Amadey Trojan

TryHackMe (Rooms: Volatility, Investigations 001 & 002) | CyberDefenders (Amadey Lab)

Overview

This report documents three memory forensics investigations completed across TryHackMe and CyberDefenders. In each case, a raw memory dump was analysed using Volatility3 to identify malicious processes, reconstruct attacker behaviour, trace C2 communications, and uncover persistence mechanisms.

Case	Platform	Malware	Outcome
Case 001	TryHackMe	Banking Trojan	Identified — Contained
Case 002	TryHackMe	WannaCry Ransomware	Post-Incident Analysis
Case 003	CyberDefenders	Amadey Trojan (APT)	Full Chain Reconstructed

Tools Used

Volatility3	Primary memory analysis framework for process, network, and DLL analysis
windows.pslist	List running processes from the doubly-linked list
windows.psscan	Scan for EPROCESS structures — catches hidden/unlinked processes
windows.pstree	Display processes with parent-child relationships
windows.netstat	Identify network connections from memory
windows.dlllist	List DLLs loaded by each process
windows.malfind	Detect injected code via RWX memory regions
windows.filescan	Scan memory for file objects — including those in malware directories
windows.handles	List open handles — used to identify mutex indicators
VirusTotal	Threat intelligence validation for IPs, domains, and hashes

Case 001 — Banking Trojan (BOB! THIS ISN'T A HORSE!)

Platform: TryHackMe | File: Investigation-1.vmem

The SOC received an alert about a quarantined endpoint suspected of a banking trojan infection. The malware was masquerading as an Adobe document. A suspicious IP (41.168.5.140) was provided as a starting point.

Step 1 — Identify the Image Profile

The first step is always to determine what OS we're working with. In Volatility3 this is automatic, but we can confirm host details with:

```
python3 vol.py -f Investigation-1.vmem windows.info
```

Build Version	2600.xpsp.080413-2111
Memory Acquired	2012-07-22 02:45:08
OS	Windows XP SP3

Step 2 — List Running Processes

Running pslist and pstree to identify all active processes and their parent-child relationships:

```
python3 vol.py -f Investigation-1.vmem windows.pslist
python3 vol.py -f Investigation-1.vmem windows.pstree
```

Among the processes, reader_sl.exe stood out immediately. It is disguised as an Adobe Reader Speed Launcher, a legitimate process — but its parent (explorer.exe) and behaviour were suspicious. Legitimate Adobe processes do not typically initiate network connections.

Suspicious Process	reader_sl.exe
PID	1640
Parent Process	explorer.exe
Parent PID	1484

Step 3 — Network Connections

Checking active network connections to confirm C2 communication:

```
python3 vol.py -f Investigation-1.vmem windows.netstat
```

An outbound connection was identified to the suspicious IP 41.168.5.140 from reader_sl.exe (PID 1640). This confirms the malware was beaconing to a command and control server.

Step 4 — User-Agent & Banking Domains

Using malfind and string extraction to identify the HTTP User-Agent used by the malware and any banking domains embedded in the process memory:

```
python3 vol.py -f Investigation-1.vmem windows.malfind
```

User-Agent	Mozilla/5.0 (Windows; U; MSIE 7.0; Windows NT 6.0; en-US)
Banking Domains	Chase Bank — confirmed (Y)
C2 IP	41.168.5.140

The malware was impersonating Internet Explorer 7 via a fake User-Agent and had Chase Bank's domain hardcoded, indicating credential harvesting targeting banking sessions.

Case 001 — Q&A Summary

Question	Answer
Build version of host machine?	`2600.xpsp.080413-2111`
Time memory file was acquired?	`2012-07-22 02:45:08`
Suspicious process?	`reader_sl.exe`
Parent process of suspicious process?	`explorer.exe`
PID of suspicious process?	`1640`
Parent process PID?	`1484`
User-Agent used by adversary?	`Mozilla/5.0 (Windows; U; MSIE 7.0; Windows NT 6.0; en-US)`
Was Chase Bank found as suspicious domain?	`Y`

Indicators of Compromise — Case 001

Indicator Type	Value
Malicious Process	`reader_sl.exe (PID 1640)`
C2 IP Address	`41.168.5.140`
Technique	`Masquerading — Adobe Reader impersonation`
Target	`Banking credentials (Chase Bank)`
User-Agent	`Mozilla/5.0 (Windows; U; MSIE 7.0; Windows NT 6.0; en-US)`

Case 002 — WannaCry Ransomware (That Kind of Hurt my Feelings)

Platform: TryHackMe | File: Investigation-2.raw

The corporation was hit by a chain of ransomware. The decryption key was already recovered. This is a post-incident analysis to identify what occurred, what actors were at play, and document the attack for the record.

Step 1 — Identify Suspicious Processes

Starting with pstree to get the full process hierarchy:

```
python3 vol.py -f Investigation-2.raw windows.pstree
```

At PID 740, the process @WanaDecryptor@ was identified — the WannaCry ransomware's decryptor interface. The process name itself is a confirmed WannaCry indicator.

Suspicious Process	`@WanaDecryptor@`
PID	`740`
Full Path	`C:\Intel\ivecuqmanpnirkt615\@WanaDecryptor@.exe`
Parent Process	`tasksche.exe`
Parent PID	`1940`

The path C:\Intel\... is notable — WannaCry uses the Intel directory as a working directory to evade detection. tasksche.exe is a fake scheduler process used by WannaCry to manage encryption tasks.

Step 2 — Identify the Malware

Cross-referencing the process name, path structure, parent process, and mutex with known threat intelligence:

```
python3 vol.py -f Investigation-2.raw windows.psscan
```

Malware Identified	`WannaCry Ransomware`
Indicator	`@WanaDecryptor@ process name + C:\Intel working directory`

Step 3 — DLL Analysis

Checking what DLLs are loaded by the decryptor process to understand its network capabilities:

```
python3 vol.py -f Investigation-2.raw windows.dlllist --pid 740
```

Socket DLL	`Ws2_32.dll (Windows Sockets API)`
Significance	`Confirms network communication capability in the ransomware`

Step 4 — Mutex Identification

Using handles to find open mutex objects — a known WannaCry indicator:

```
python3 vol.py -f Investigation-2.raw windows.handles --pid 740
```

Mutex	`MsWinZonesCacheCounterMutexA`
Significance	`Known WannaCry mutex — confirms malware identity beyond doubt`

Step 5 — Files in Malware Working Directory

To enumerate all files loaded from the WannaCry working directory:

```
python3 vol.py -f Investigation-2.raw windows.filescan | grep ivecuqmanpnirkt615
```

Plugin Used	windows.filescan
Purpose	Identify all file objects present in the malware's working directory

Case 002 — Q&A Summary

Question	Answer
Suspicious process at PID 740?	**@WanaDecryptor@**
Full path of suspicious binary at PID 740?	**C:\Intel\ivecuqmanpnirkt615\@WanaDecryptor@.exe**
Parent process of PID 740?	**tasksche.exe**
Suspicious parent process PID?	**1940**
Malware present on the system?	**WannaCry**
DLL loaded for socket creation?	**Ws2_32.dll**
Known mutex indicator?	**MsWinZonesCacheCounterMutexA**
Plugin to identify files in working directory?	**windows.filescan**

Indicators of Compromise — Case 002

Indicator Type	Value
Malicious Process	@WanaDecryptor@.exe (PID 740)
Full Path	C:\Intel\ivecuqmanpnirkt615\@WanaDecryptor@.exe
Fake Parent Process	tasksche.exe (PID 1940)
Socket DLL	Ws2_32.dll
Mutex	MsWinZonesCacheCounterMutexA
Malware Family	WannaCry Ransomware

Case 003 — Amadey Trojan (APT-C-36)

Platform: CyberDefenders | Lab: Amadey — APT-C-36

An EDR alert flagged suspicious activity on a Windows workstation after hours. The malware was identified as consistent with Amadey Trojan Stealer behaviour. A memory dump was provided for full post-compromise analysis to reconstruct the infection chain.

Step 1 — Identify the Malicious Process

Starting with pstree to map the process hierarchy and identify anomalies:

```
python3 vol.py -f amadey.mem windows.pstree
```

The process lssass.exe was immediately suspicious — it is masquerading as the legitimate Windows process lsass.exe (Local Security Authority Subsystem) by adding an extra ‘s’. This is a classic masquerading technique.

Malicious Process	lssass.exe (masquerading as lsass.exe)
Execution Path	C:\Users\0XSH3R~1\AppData\Local\Temp\925e7e99c5\lssass.exe
Location	Temp directory — typical malware staging area

Step 2 — C2 Communication

Running netstat to identify active outbound connections from the malicious process:

```
python3 vol.py -f amadey.mem windows.netstat
```

C2 Server IP	41.75.84.12
Significance	Confirmed external C2 communication from lssass.exe

Step 3 — Payload Downloads

Using filescan to identify files being fetched from the C2 server:

```
python3 vol.py -f amadey.mem windows.filescan
```

Files Downloaded	2 distinct files
Key Payload Path	C:\Users\0xSh3rl0ck\AppData\Roaming\116711e5a2ab05\clip64.dll
Payload Type	DLL — clipboard stealer module

Step 4 — Payload Execution via LOLBin

Identifying how the downloaded DLL was executed. Amadey abuses legitimate Windows binaries (LOLBins) to avoid detection:

```
python3 vol.py -f amadey.mem windows.pstree
```

Child Process	rundll32.exe
Purpose	Executes clip64.dll using the legitimate Windows rundll32.exe (LOLBin abuse)
Technique	T1218.011 — Signed Binary Proxy Execution: Rundll32

Step 5 — Persistence Mechanism

Checking for scheduled tasks as an additional persistence location beyond the Startup folder:

```
python3 vol.py -f amadey.mem windows.filescan | grep Tasks
```

Persistence Location 1	Windows Startup folder
Persistence Location 2	C:\Windows\System32\Tasks\lssass.exe
Technique	T1053.005 — Scheduled Task / Job

Full Amadey Infection Chain

- Execution: lssass.exe launched from Temp directory (masquerading as lsass.exe)
- C2 Communication: Beaconed to 41.75.84.12 to receive commands
- Payload Delivery: Downloaded 2 files including clip64.dll from C2 server
- LOLBin Execution: rundll32.exe used to execute the downloaded DLL
- Persistence: Scheduled Task created at C:\Windows\System32\Tasks\lssass.exe

Case 003 — Q&A Summary

Question	Answer
Parent process triggering malicious behaviour?	**lssass.exe**
Full path of malicious process?	**C:\Users\0XSH3R~1\AppData\Local\Temp\925e7e99c5\lssass.exe**
C2 server IP?	**41.75.84.12**
Number of distinct files downloaded?	**2**
Full path of downloaded payload?	**C:\Users\0xSh3rl0ck\AppData\Roaming\116711e5a2ab05\clip64.dll**
Child process executing the payload?	**rundll32.exe**
Additional persistence location?	**C:\Windows\System32\Tasks\lssass.exe**

Indicators of Compromise — Case 003

Indicator Type	Value
Malicious Process	lssass.exe (masquerading as lsass.exe)
Execution Path	C:\Users\0XSH3R~1\AppData\Local\Temp\925e7e99c5\lssass.exe
C2 IP Address	41.75.84.12
Downloaded Payload	C:\Users\0xSh3rl0ck\AppData\Roaming\116711e5a2ab05\clip64.dll
LOLBin Abused	rundll32.exe
Persistence	C:\Windows\System32\Tasks\lssass.exe (Scheduled Task)
Technique	Masquerading + C2 + Payload Delivery + LOLBin + Scheduled Task

Investigation Summary — All Cases

Case	Malware	Key Process	C2 IP	Key Technique
001	Banking Trojan	reader_sl.exe	41.168.5.140	Masquerading + C2
002	WannaCry	@WanaDecryptor@	N/A	Ransomware + Mutex
003	Amadey Trojan	lssass.exe	41.75.84.12	LOLBin + Sched. Task

Skills Demonstrated

- Memory Forensics — Analysed .vmem and .raw memory dumps using Volatility3 across 3 real-world scenarios
- Process Analysis — Used pslist, psscan, and pstree to identify suspicious and masquerading processes
- Masquerading Detection — Identified reader_sl.exe (Adobe) and lssass.exe (lsass) impersonation techniques
- Network Forensics — Traced C2 connections using windows.netstat and cross-referenced with VirusTotal
- DLL Analysis — Used dlllist to identify socket creation DLLs and malicious payload modules
- Mutex Analysis — Identified WannaCry's known mutex (MsWinZonesCacheCounterMutexA) via handles plugin
- LOLBin Detection — Identified rundll32.exe abuse for payload execution (T1218.011)
- Persistence Hunting — Discovered Scheduled Task persistence in C:\Windows\System32\Tasks
- Attack Chain Reconstruction — Mapped full infection chains for all three malware families
- MITRE ATT&CK Mapping — Correlated findings to relevant techniques (T1036, T1053, T1218)

TryHackMe — Volatility Room | CyberDefenders — Amadey Lab | Memory Forensics / DFIR